

Contractor-Executable Roadmap for Unity + FishNet

Attribute	Detail
Version	1.0
Related	GDD.md (current single-player behavior)
Stakeholder Ask	FishNet multiplayer · Inventory system · Colleague + enemy characters · Smarter enemy AI
Engine	Unity (existing project)
Networking	FishNet (LTS-compatible version)
Default Mode	Co-op PvE · 2–4 players · Listen-server host · PC-first

TABLE OF CONTENTS

1. Purpose & how to use this document
2. Scope alignment
3. Assumptions & open decisions
4. Phase 0 — Discovery & technical baseline
5. Phase 1 — FishNet multiplayer foundation
6. Phase 2 — Board & session synchronization
7. Phase 3 — Gameplay authority & turn rules
8. Phase 4 — Inventory system (production)
9. Phase 5 — New characters: colleagues & enemies
10. Phase 6 — Enemy AI upgrade
11. Phase 7 — QA, performance & platform hardening

12. Deliverables checklist per phase

13. Risk register

1. PURPOSE & HOW TO USE THIS DOCUMENT

This plan turns the current single-player codebase into a contractor-executable roadmap. Each phase lists:

Goals	What "done" means for the client.
Functions / Components	Major Unity scripts, services, or FishNet roles to add or refactor.
Tasks	Ordered work units.
Dependencies	What must exist before starting.
Outputs	Artifacts the client can review (builds, docs, demos).

2. SCOPE ALIGNMENT

2.1 Full stakeholder list (original message)

- Multiplayer networking (prefer FishNet).
- Inventory system (beyond today's GameManager + UI slots).
- Characters: colleagues (allies / NPCs) and additional enemy types.
- Enemy AI upgrade (smarter than current greedy one-step logic).

2.2 Phased vs. "multiplayer only"

Some conversations narrowed Phase A to multiplayer only. This document orders work so that:

- **Phases 0–3** deliver playable networked sessions with core rules preserved.
- **Phases 4–6** add inventory, characters, and AI without rewriting net foundations.

If the contract is multiplayer-only, stop after Phase 3 (plus minimal Phase 7 for release quality).

3. ASSUMPTIONS & OPEN DECISIONS

Decisions below should be confirmed in writing before Phase 1 coding:

ID	Decision	Options (examples)
D1	Game mode	Co-op PvE (all vs enemies) vs PvP last-survivor
D2	Player count	2, 4, or scalable
D3	Host model	Listen-server host vs dedicated headless server

D4	Platforms	PC first; WebGL / mobile later (affects transport)
D5	Matchmaking	None (IP / host code) vs Steam / UGS / PlayFab
D6	Colleague control	AI-only ally vs extra local / network player

Default assumption (if client silent): Co-op PvE, 2–4 players, listen-server host, PC first, no matchmaking, AI colleagues initially.

PHASE 0

Discovery & Technical Baseline

■ Goals

- Eliminate unknowns that would invalidate estimates (board sync model, turn rules with multiple humans).
- Freeze a baseline build that reproduces current single-player behavior.

■ Functions / Components (audit only)

Component	Responsibility
GameManager	All fields that must become network-authoritative vs. client UI-only.
BoardManager	Every random spawn rule; identify RNG seed strategy.
Player / Enemy	State vectors per instance (HP, food, stun, speed, flags).
Wall / Chests	Events that must replicate (open, loot assignment).
Traps / Bombs	Trigger ownership and one-shot state.

■ Tasks

- 1 Playthrough matrix** — Run levels 1, 2, 5, 8, 10; record video or checklist for edge cases (exit lock, altar, bosses).
- 2 State matrix** — Spreadsheet: Variable | Owner (future) | Sync method | Notes.
- 3 Refactor hotspots** — List FindObjectOfType / tag scans to replace with injected references or grid queries in Phase 2–3.
- 4 FishNet version pin** — Choose LTS-compatible FishNet version for current Unity editor; record in readme.

■ Dependencies

- Unity project opens cleanly; BoardScene runs.

■ Outputs

- Technical discovery memo (3–5 pages): state matrix + open decisions D1–D6 answered or flagged.
- Baseline tag in version control: *pre-multiply* (optional but recommended).

PHASE 1

FishNet Multiplayer Foundation

■ Goals

- Clients can connect to a host/server, spawn networked player prefabs, and remain in sync at idle (connection lifecycle only).

■ Functions / Components to introduce

Component	Responsibility
NetworkManager (FishNet)	Transport, scene handling, player spawning hooks.
Connection / Session Config	Max clients, timeouts, NAT notes for PC builds.
Networked Player Shell	Empty or minimal NetworkBehaviour prefab replacing solo Player for connection tests.
Bootstrap Scene (optional)	Menu flow: Host / Join / IP entry.

■ Tasks

- 1** Import FishNet; resolve assembly conflicts; add **NetworkManager** to bootstrap scene.
- 2** Create **PlayerNetworkRoot** prefab with NetworkObject + minimal sync (e.g., nameplate or ping).
- 3** Implement **Host** and **Client** entry from UI (placeholder OK).
- 4** Verify spawn / despawn on disconnect; no gameplay yet.

■ Dependencies

- Phase 0 complete.

■ Outputs

- Build where 2 instances connect and see each other's spawned networked objects (proof of life).

PHASE 2

Board & Session Synchronization

■ Goals

- One procedural board exists per stage authoritatively; all clients see the same layout (walls, floor, interactables).

■ Functions / Components

Component	Responsibility
BoardDirector (new)	Server-only: runs generation, owns seed, triggers spawns.
BoardSync / Spawn Bus	Server instantiates networked prefabs for tiles / entities OR sends compressed layout blob.
Deterministic RNG	Single System.Random or Unity.Mathematics stream from stage seed (level + run id).
GridModel (new, recommended)	2D array: walkable, occupant id, reference to NetworkObject if any.

■ Tasks

- 1 Extract generation from **BoardManager.SetupScene** into **BoardDirector.Generate(level, seed)** callable only on server.
- 2 Choose sync strategy (pick one): **(A)** Network prefabs per entity (wall, food, chest, trap) — simple, more bandwidth. **(B)** Tilemap + delta spawn list — fewer objects, more code.
- 3 Replicate seed in **RunState** NetworkBehaviour so late joiners are out-of-scope or disallowed until next run (product decision).
- 4 Replace tag scans for "closest food" inputs with **GridModel** queries on server (prepares AI Phase 6).

■ Dependencies

- Phase 1 working connection.

■ Outputs

- Two clients walk (movement can be stub) on identical board geometry; no desync on wall positions in test harness.

PHASE 3

Gameplay Authority & Turn Rules

■ Goals

- Full game loop works in multiplayer: player moves, enemies move, exit rule, damage / food, no cheats from client.

■ Functions / Components

Component	Responsibility
RunController (NetworkBehaviour)	Server: playersTurn, enemiesMoving, stage transitions; mirrors today's GameManager loop.
PlayerController (NetworkBehaviour)	Client sends MoveIntent (dx, dy); server validates; plays animation locally predicted if desired (optional).
EnemyController (NetworkBehaviour)	Server runs MoveEnemy equivalent; replicates state snapshots or grid position.
CombatService (server)	Centralizes food steal, HP change, stun to avoid duplicated logic.
UI Façade	Client reads sync vars / Observers for HP, food, inventory display.

■ Tasks

- 1 Move turn FSM to server: only server advances **MoveEnemies** coroutine.
- 2 Implement RPC or Target pattern for player intent; reject illegal moves (wall, wrong turn).
- 3 Replicate enemy list as NetworkObject collection or id registry.
- 4 Implement exit gating on server: **allEnemiesDead** authoritative bool.
- 5 Port Game Over / stage advance as server-driven scene events (FishNet scene events or custom load).
- 6 **Analytics** — Decide host-only vs. all clients; avoid double-firing.

■ Dependencies

- Phase 2 board sync stable.

■ Outputs

- 2+ human players can complete a stage cooperatively (if D1 = co-op); enemies behave as in GDD; disconnect handled gracefully (host migration out of scope unless requested).

PHASE 4

Inventory System (Production)

■ Goals

- Replace GameManager icon lists + stub Inventory with a data-driven, networked inventory.

■ Functions / Components

Component	Responsibility
ItemDefinition (ScriptableObject)	Id, display name, icon, max stack, use handler id.
InventoryContainer (NetworkBehaviour)	Slots array, server-authoritative mutations.
ItemUseRouter (server)	Maps handler id → heal / speed buff / key / quest item.
InventoryUI	Binds to container SyncList / Observers; no direct GameManager loot flags.
Migration Adapters	Chest rolls produce ItemInstance instead of touching hasHealthPotion.

■ Tasks

- 1 Define item catalog for existing loot: speed +1 perm, max HP, HP potion, speed potion (+ future slots).
- 2 Implement pickup / use RPCs with validation (range, turn, ownership).
- 3 Migrate chest Wall logic to grant items into **InventoryContainer**.
- 4 Deprecate **hasHealthPotion** / **hasSpeedPotion** booleans behind façade; remove when safe.
- 5 Save / load (if needed): PlayerPrefs or run-end summary — product decision.

■ Dependencies

- Phase 3 combat authority (items affect stats on server).

■ Outputs

- Design doc appendix: item table; test cases for duplicate chest, full inventory, use on wrong turn.

PHASE 5

New Characters — Colleagues & Enemies

■ Goals

- Colleagues exist as distinct actors with faction and AI or player control (per D6).

- New enemy types via data, not copy-paste prefabs only.

■ Functions / Components

Component	Responsibility
Faction Enum	Player, Ally, Enemy, Neutral.
CharacterProfile (ScriptableObject)	Base speed, damage, loot table ref, AI profile id.
AllyBrain (server)	Utility AI or follow-player leash; no friendly fire (unless designed).
EnemyVariant	Prefab + profile; spawned from weighted tables in BoardDirector.
SpawnTable (per level band)	Replaces single enemyTiles random pick for late game.

■ Tasks

- 1 Add Ally networked prefab; host spawns with player group; collision rules updated (**OnCantMove** targets).
- 2 Define 1 colleague archetype (e.g., "scavenger" — seeks food, avoids traps).
- 3 Add 2 enemy archetypes (e.g., bruiser — high HP; skirmisher — fast steal).
- 4 Update dialogue hooks for boss intros if new bosses requested (optional).
- 5 **Balance:** ensure solo survivor exit rule still makes sense with allies (e.g., exit when no enemy faction remains).

■ Dependencies

- Phase 3–4 stable (stats + items feed characters).

■ Outputs

- Character bible markdown: per-archetype stats, behavior summary, spawn tiers.

PHASE 6

Enemy AI Upgrade

■ Goals

- Enemies pursue and flank credibly; fewer oscillations and blocked-axis stalls.

■ Functions / Components

Component	Responsibility
IEnemyBrain interface	ChooseStep(GridModel, self, blackboard) → (dx, dy)
BrainGreedyLegacy	Current behavior preserved for A/B or easy enemies.
BrainPathfind	BFS / A* on GridModel with dynamic obstacles.
Blackboard	Last-seen player cell, noise events, trap memory (optional).
DifficultyDirector	Picks brain + lookahead depth by level and enemy tier.

■ Tasks

- 1 Implement pathfinding to current target cell (food → chest → player per existing priority).
- 2 Add tie-break when multiple same-length paths (spread, not ping-pong).
- 3 Local avoidance: if goal cell occupied, try lateral detour with cost.
- 4 Boss-only lookahead (optional): 1-ply simulation for deadly moves.
- 5 **Performance budget:** cap path calls per frame on server; stagger enemy think steps.

■ Dependencies

- GridModel from Phase 2 operational on server.

■ Outputs

- AI test maps: maze, chokepoint, open field — metrics: time-to-reach-food, player pressure index.

PHASE 7

QA, Performance & Platform Hardening

■ Goals

- Ship-quality stability for agreed platforms; document limits.

■ Functions / Components

Component	Responsibility
Automated Test Scene	Bot players spam moves; soak test 30+ minutes.
Net Stats HUD (dev)	RTT, tick rate, snapshot sizes.
Logging	Structured log for desync reports (seed, tick, action queue).

■ Tasks

- 1 **Stress:** 4 players, max enemies, profile CPU on host.
- 2 **Packet loss simulation** (FishNet tools or Clumsy) — verify recovery.
- 3 **WebGL / mobile** (if in scope): transport validation; input rework.
- 4 **Privacy:** Analytics toggle; compliance note for client.

■ Dependencies

- Feature-complete per contract.

■ Outputs

- Release notes, known issues, server hosting quickstart (if dedicated).

12. DELIVERABLES CHECKLIST PER PHASE

Phase	Client-Visible Deliverable
0	Discovery memo + decisions D1–D6
1	2-client connection demo
2	Identical board on all peers
3	Full co-op loop playable
4	Inventory catalog + UI
5	New ally + 2 enemy types in spawn tables
6	Pathfind AI + benchmark scenes
7	Release candidate build + notes

13. RISK REGISTER

Severity	Risk	Impact	Mitigation
HIGH	Scope says "multiplayer only" but code needs board refactor	Schedule slip	Phase 0 sign-off on sync strategy.
MED	WebGL + FishNet limitations	Blocked platform	PC-first; WebGL spike in Phase 0.
HIGH	GameManager monolith	Bugs during port	Extract RunController early in Phase 3.
MED	AI CPU on host	Lag	Staggered thinking; simpler brains for grunts.
MED	No GDD from client	Rework	Phase 5 "Character bible" requires written approval per archetype.